IACR Transactions on Cryptographic Hardware and Embedded Systems
ISSN 2569-2925, Vol. 2026, No. 2, pp. 272–295. DOI:10.46586/tches.v2026.i2.272-295

A Framework for designing High-Order Side-Channel Protected Hardware Implementations of ML-KEM

Eros Camacho-Ruiz[1], Pablo Navarro-Torrero[1] and Alejandro Cabrera Aldaya[2]

[1]Instituto de Microelectrónica de Sevilla (IMSE-CNM), CSIC / US, Sevilla, Spain
[2]Tampere University, Tampere, Finland
camacho@imse-cnm.csic.es

Abstract. ML-KEM (formerly Kyber) has recently been adopted as FIPS 203 in the NIST Post-Quantum Cryptography standardization process. While existing hardware implementations primarily optimize for performance, they often lack protections against side-channel attacks. We introduce `HOPE-MLKEM`, a framework that includes the first configurable, open, and full-hardware implementation of ML-KEM with integrated high-order protection against timing and power side-channel attacks. Our modular architecture supports all security levels and operations, incorporating optimized building blocks for polynomial arithmetic, modular multiplication, and programmable control logic. At the same time, this methodology enables the seamless integration of masking countermeasures up to high orders. We evaluated `HOPE-MLKEM` on FPGA and ASIC platforms, achieving competitive results compared to state-of-the-art unprotected designs while providing resistance against high-order side-channel attacks. Beyond its technical contributions, `HOPE-MLKEM` is released as an open-source framework to foster community-driven exploration of design choices, leakage evaluation, and hardware optimizations.

Keywords: PQC · ML-KEM · Hardware · Open-source · Countermeasures

1 Introduction

The standardization of Post-Quantum Cryptography (PQC) [Nat24] has triggered a global push toward efficient and secure implementations of selected schemes. Among them, the lattice-based key encapsulation mechanism ML-KEM (formerly Kyber) was standardized in FIPS 203 [MLK24] for its strong balance between performance and security. With ML-KEM set to become a core component of next-generation secure communication systems, the development of efficient and robust implementations is of the greatest importance.

On the software side, `pqm4` [KRSS19] has emerged as the de facto benchmarking framework for PQC on ARM Cortex-M4 devices. However, cryptographic software implementations on embedded platforms could remain vulnerable to Side-Channel Attacks (SCA), where adversaries exploit physical leakages, such as power consumption, to recover secrets. Although several first- and higher-order countermeasures have been proposed [BGR+21, HKL+22] the additional latency and memory usage they require is often too high for resource-constrained environments. This trade-off highlights the indispensable role of dedicated hardware in achieving practical, scalable, and SCA resilient deployments.

Existing hardware implementations of ML-KEM have mainly prioritized performance [NKK+23, KJSL24], with only recent efforts beginning to consider side-channel protections [MJB+24, XWT25]. However, these works either limit themselves to first-order

Licensed under Creative Commons License CC-BY 4.0.
Received: 2025-10-15 Accepted: 2025-12-15 Published: 2026-04-23

security or rely on specialized design choices that fail to scale to higher orders. The feasibility of implementing robust higher-order protections in hardware without prohibitive area or performance overheads remains an open challenge. This gap is especially pressing for embedded and IoT applications, where security, efficiency, and scalability are critical.

We present `HOPE-MLKEM`, a framework that includes the first full hardware implementation of ML-KEM, achieving scalable, high-order side-channel resistance with competitive performance and resource efficiency. **Our contributions** are as follows:

- **Modular and flexible hardware design.** A unified architecture supporting all ML-KEM security levels ($k = 2, 3, 4$) and all KEM operations (Key Generation, Encapsulation, Decapsulation), extensible for systematic exploration of design choices.
- **Optimized building blocks.** Novel architectures for NTT / INTT and PWM operations, modular multipliers, and programmable control logic with a compact instruction set, enabling performance beyond state-of-the-art hardware designs.
- **Scalable high-order protection.** The framework integrates practical masking techniques scalable to higher orders, providing a flexible trade-off between side-channel resistance, performance, and resource utilization. In particular, our design includes countermeasures against both timing and power side-channel attacks.
- **Open-source framework.** We release an open-source and reconfigurable framework for further research on high-order protections and hardware optimizations, with FPGA / ASIC flows, simulation and leakage evaluation tools. Available at https://github.com/HWSec-CSIC/hope-mlkem.

This paper is organized as follows. Section 2 provides an overview of the algorithms specified in FIPS 203 for ML-KEM. Section 3 describes the proposed hardware architecture, detailing the design flow and the main modules. Section 4 discusses the side-channel countermeasures implemented and their integration into the framework. Section 5 presents the experimental results, analyzing both the performance and the security aspects. Finally, Section 6 concludes the paper and outlines future research directions.

2 The ML-KEM Scheme

ML-KEM is a lattice-based Key Encapsulation Mechanism (KEM) standardized in FIPS 203 [MLK24]. Its security is based on the hardness of the Module-LWE problem. The underlying design can be seen as a CPA-secure public-key encryption scheme, made a IND-CCA2-secure KEM through the Fujisaki-Okamoto transform. It operates on the polynomial ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$, where $q = 3329$ and $n = 256$.

Algorithms 1, 2 and 3 show the key generation, encapsulation, and decapsulation algorithms as in FIPS 203. These algorithms correspond to the `_internal` algorithms in FIPS 203, however for the sake of illustration we remove this suffix. ML-KEM supports three parameter sets from lowest to highest security: ML-KEM-512 ($K = 2, \eta_1 = 3, \eta_2 = 2, d_u = 10, d_v = 4$), ML-KEM-768 ($K = 3, \eta_1 = 2, \eta_2 = 2, d_u = 10, d_v = 4$), and ML-KEM-1024 ($K = 4, \eta_1 = 2, \eta_2 = 2, d_u = 11, d_v = 5$).

When observed from a high-level perspective, it is possible to decompose the ML-KEM algorithm into four primary classes of operations based on its functionality. This decomposition will help to understand the proposed hardware design. The first class encompasses a selection of Keccak-based cryptographic primitives, namely SHA3-256, SHA3-512, SHAKE-128, and SHAKE-256, denoted by $\mathsf{H}(\cdot)$, $\mathsf{G}(\cdot)$, $\mathsf{XOF}(\cdot)$, and $\mathsf{PRF}(\cdot)$ or $\mathsf{J}(\cdot)$, respectively. These functions are used in conjunction with sampler functions class $\mathsf{SampleNTT}(\cdot)$ and $\mathsf{SamplePolyCBD}(\cdot)$ to generate the public matrix $\hat{\mathbf{A}}$, the secret and error vectors $\mathbf{s}$ and $\mathbf{e}$, as well as the encryption specific vectors $\mathbf{y}$, $\mathbf{e_1}$ and e_2. Another class comprises polynomial domain transformations and arithmetic operations, including the Number Theoretic Transform (NTT) and its inverse (INTT), the Point-Wise Multiplication (PWM) denoted as "$\circ$", addition (ADD) denoted as "$+$", and subtraction (SUB) denoted

Algorithm 1 $(\mathsf{ek}, \mathsf{dk}) = \texttt{ML-KEM.KeyGen}(d, z)$

Uses randomness, $d \in \mathbb{B}^{32}$ and $z \in \mathbb{B}^{32}$ to generate the encapsulation key $ek \in \mathbb{B}^{384k+32}$ and the decapsulation key $dk \in \mathbb{B}^{768k+96}$.

$(\mathsf{ek}_{\text{PKE}}, \mathsf{dk}_{\text{PKE}}) = \texttt{K-PKE.KeyGen}(d)$:

1: $(\rho, \sigma) \leftarrow \mathsf{G}(d \| k)$
2: $\hat{\mathbf{A}}[i,j] \leftarrow \mathsf{SampleNTT}(\rho \| j \| i)\Big|_{i<k, j<k}$
3: $\mathbf{s}[i] \leftarrow \mathsf{SamplePolyCBD}_{\eta_1}(\mathsf{PRF}_{\eta_1}(\sigma, i))\big|_{i<k}$; $\mathbf{e}[i] \leftarrow \mathsf{SamplePolyCBD}_{\eta_1}(\mathsf{PRF}_{\eta_1}(\sigma, i+k))\big|_{i<k}$
4: $\hat{\mathbf{s}} \leftarrow \mathsf{NTT}(\mathbf{s})$; $\hat{\mathbf{e}} \leftarrow \mathsf{NTT}(\mathbf{e})$
5: $\hat{\mathbf{t}} \leftarrow \hat{\mathbf{A}} \circ \hat{\mathbf{s}} + \hat{\mathbf{e}}$
6: $\mathsf{ek}_{\text{PKE}} \leftarrow \mathsf{ByteEncode}_{12}(\hat{\mathbf{t}}) \| \rho$; $\mathsf{dk}_{\text{PKE}} \leftarrow \mathsf{ByteEncode}_{12}(\hat{\mathbf{s}})$
7: $\mathsf{ek} \leftarrow \mathsf{ek}_{\text{PKE}}$; $\mathsf{dk} \leftarrow (\mathsf{dk}_{\text{PKE}} \| \mathsf{ek} \| \mathsf{H}(\mathsf{ek}) \| z)$

Algorithm 2 $(K, c) = \texttt{ML-KEM.Encaps}(\mathsf{ek}, m)$

Uses the encapsulation key, $ek \in \mathbb{B}^{384k+32}$ and randomness, $m \in \mathbb{B}^{32}$ to generate a key, $K \in \mathbb{B}^{32}$ and an associated ciphertext, $c \in \mathbb{B}^{32(d_u k + d_v)}$

1: $(K, r) \leftarrow \mathsf{G}(m \| \mathsf{H}(\mathsf{ek}))$

$c = \texttt{K-PKE.Encrypt}(\mathsf{ek}_{\text{PKE}}, m, r)$:

2: $\mu \leftarrow \mathsf{Decompress}_1(\mathsf{ByteDecode}_1(m))$; $\hat{\mathbf{t}} = \mathsf{ByteDecode}_{12}(\mathsf{ek}_{\text{PKE}}[0 : 384k])$
3: $\hat{\mathbf{A}}[i,j] \leftarrow \mathsf{SampleNTT}(\rho \| j \| i)\Big|_{i<k, j<k}$
4: $\mathbf{y}[i] \leftarrow \mathsf{SamplePolyCBD}_{\eta_1}(\mathsf{PRF}_{\eta_1}(r, i))\big|_{i<k}$
5: $\mathbf{e_1}[i] \leftarrow \mathsf{SamplePolyCBD}_{\eta_2}(\mathsf{PRF}_{\eta_2}(r, i+k))\big|_{i<k}$; $e_2 \leftarrow \mathsf{SamplePolyCBD}_{\eta_2}(\mathsf{PRF}_{\eta_2}(r, 2k))$
6: $\mathbf{u} \leftarrow \mathsf{INTT}(\hat{\mathbf{A}}^\top \circ \mathsf{NTT}(\mathbf{y})) + \mathbf{e_1}$; $v \leftarrow \mathsf{INTT}(\hat{\mathbf{t}}^\top \circ \mathsf{NTT}(\mathbf{y})) + e_2 + \mu$
7: $c_1 \leftarrow \mathsf{ByteEncode}_{d_u}(\mathsf{Compress}_{d_u}(\mathbf{u}))$; $c_2 \leftarrow \mathsf{ByteEncode}_{d_v}(\mathsf{Compress}_{d_v}(v))$
8: $c \leftarrow (c_1 \| c_2)$

Algorithm 3 $(K, c) = \texttt{ML-KEM.Decaps}(\mathsf{dk}, c)$

Uses the decapsulation key, $dk \in \mathbb{B}^{768k+96}$, to produce a shared secret key, $K' \in \mathbb{B}^{32}$ from a ciphertext, $c \in \mathbb{B}^{32(d_u k + d_v)}$.

1: $\mathsf{dk}_{\text{PKE}} = \mathsf{dk}[0 : 384k]$; $\mathsf{ek}_{\text{PKE}} = \mathsf{dk}[384k : 768k + 32]$
2: $h = \mathsf{dk}[768k + 32 : 768k + 64]$; $z = \mathsf{dk}[768k + 64 : 768k + 96]$

$m' = \texttt{K-PKE.Decrypt}(\mathsf{dk}_{\text{PKE}}, c)$:

3: $c_1 \leftarrow c[0 : 32 d_u k]$; $c_2 \leftarrow c[32 d_u k : 32(d_u k + d_v)]$
4: $\mathbf{u}' \leftarrow \mathsf{Decompress}_{d_u}(\mathsf{ByteDecode}_{d_u}(c_1))$; $v' = \mathsf{Decompress}_{d_v}(\mathsf{ByteDecode}_{d_v}(c_2))$
5: $\hat{\mathbf{s}} \leftarrow \mathsf{ByteDecode}_{12}(\mathsf{dk}_{\text{PKE}})$
6: $w \leftarrow v' - \mathsf{INTT}(\hat{\mathbf{s}}^\top \circ \mathsf{NTT}(\mathbf{u}'))$
7: $m' \leftarrow \mathsf{ByteEncode}_1(\mathsf{Compress}_1(w))$
8: $(K', r') \leftarrow G(m' \| h)$
9: $\bar{K} \leftarrow J(z \| c)$
10: $c' \leftarrow \texttt{K-PKE.Encrypt}(\mathsf{ek}_{\text{PKE}}, m', r')$
11: **if** $c = c'$ **then** $K \leftarrow K'$ **else** $K \leftarrow \bar{K}$

as "$-$" over $\mathbb{Z}_q$. $\mathsf{Compress}_d$ and $\mathsf{Decompress}_d$ functions will transform $\mathbb{Z}_q \to \mathbb{Z}_{2^d}$ and $\mathbb{Z}_{2^d} \to \mathbb{Z}_q$ respectively. Lastly, the functions $\mathsf{ByteEncode}_d$ and $\mathsf{ByteDecode}_d$ map integers to arrays of bytes and arrays of bytes to integers, respectively. For notation, we write the vectors in $\left(\mathbb{Z}_q^{256}\right)^k$ in **boldface**, while vectors in $\mathbb{Z}_q^{256}$ are written in plain font. Following FIPS 203, we use the symbol "$\hat{}$" to denote vectors in T_q, the image of R_q under the NTT.

3 Hardware Design

Designing an efficient hardware architecture for ML-KEM requires a modular structure to maximize hardware reuse and enable configurability across different security levels and operating modes. Our proposed top-level architecture, shown in Figure 1, follows this principle. This unit encompasses four main computational modules: the *Butterfly Unit* (`BU`), the *Centered Binomial Distribution* module (`CBD`), the *Rejection Sampler* (`RS`), and the `Keccak` module. These components are selectively activated and tightly coordinated through a dedicated scheduling strategy. Data input and output are handled by specialized `Encoding + Compression` and `Decoding + Decompression` modules.

The overall architecture is controlled by two main control programs, each stored in a dedicated ROM: one managing the `Keccak`–`CBD`–`RS` datapath, and the other controlling the `BUs`. Both programs are executed by FSMs that coordinate the operational flow based on the selected mode. The control signal is divided into the operation (`op`) and the security mode (`mode`). The `op` bits trigger key generation (`01`), encapsulation (`10`), or decapsulation (`11`). The `mode` selects the ML-KEM parameter set: 512 (`01`), 768 (`10`), or 1024 (`11`). The value `00` is reserved for both fields. By simply adjusting these values, the module can be dynamically configured to support various security levels and operational modes.

On the left side of Figure 1, the `Keccak` datapath receives its inputs through an input selector that manages elements such as the public key (ek), ciphertext (c), and message (m), while also generating the required cryptographic seeds, $H(ek)$, ρ, σ, r, and the shared secret (K). The `Keccak` core interacts with the `CBD` and `RS` modules to produce the necessary polynomial representations, all coordinated by the `Keccak`+`CBD`+`RS` control unit. On the right side, the `BUs` are controlled by a dedicated control program. The *Data Management Unit* (`DMU`) is responsible for controlling data flow among multiple `BUs`, regulating access to shared RAM banks, and optimizing scheduling in multi-unit configurations.

This section details the design of the individual building blocks that compose the modular methodology. Each building block can be configured according to a scheduling strategy controlled by a specific set of instructions. To protect against timing attacks, all operations are constant-time, eliminating data-dependent control flow and memory access patterns. The design and its optimizations were conceived to easily integrate side-channel countermeasures against power attacks, as outlined in Section 4.

3.1 Modular methodology

The proposed architecture combines recursive functionalities into a small set of reusable modules. Specifically, a dedicated `Keccak` module handles all hash computations, a `CBD` module performs SamplePolyCBD($\cdot$), and a `RS` module implements SampleNTT($\cdot$). In

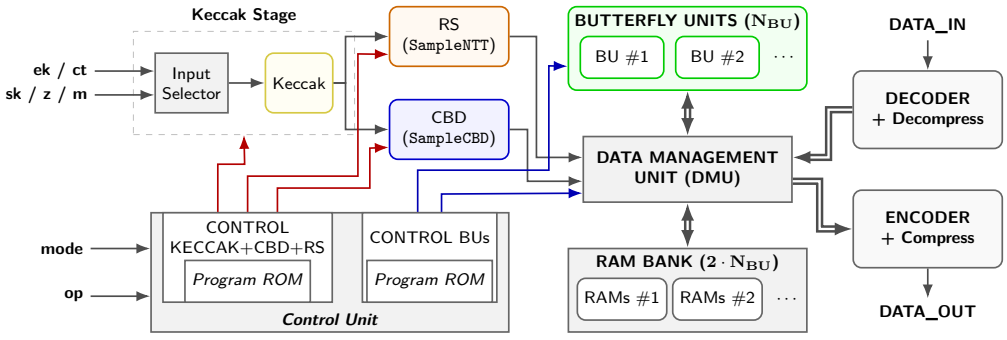


Figure 1: Top design of ML-KEM module. Black arrows represent datapath, blue one represents the control logic for `Keccak`+`CBD`+`RS`, red one represents the control for the `BUs` and the `DMU`.

addition, the BU executes all polynomial arithmetic on $\mathbb{Z}_q$, including NTT, INTT, PWM, ADD, and SUB. HOPE-MLKEM includes all KEM operations, although for illustration purposes, Figure 2 shows the interaction of the hardware modules during the K-PKE, since it is the most extensive operation in the KEM: K-PKE.KeyGen, K-PKE.Encrypt, and K-PKE.Decrypt for ML-KEM-512. Focusing on K-PKE.KeyGen (Figure 2a), the process begins with the provision of randomness d, from which the seeds ρ and σ are derived through $\mathsf{G}(d, k)$, using Keccak module. Following Algorithm 1, the next step is the computation of the matrix $\hat{\mathbf{A}}$, which requires $\mathsf{SampleNTT}(\cdot)$ implemented through the RS module using Keccak outputs. Subsequently, the secret vector $\hat{\mathbf{s}}$ and the error vector $\hat{\mathbf{e}}$ are generated through the NTT within the BU, driven by CBD outputs. Finally, the BU performs the PWM and the addition ADD with the error vector to produce the output vector $\hat{\mathbf{t}}$.

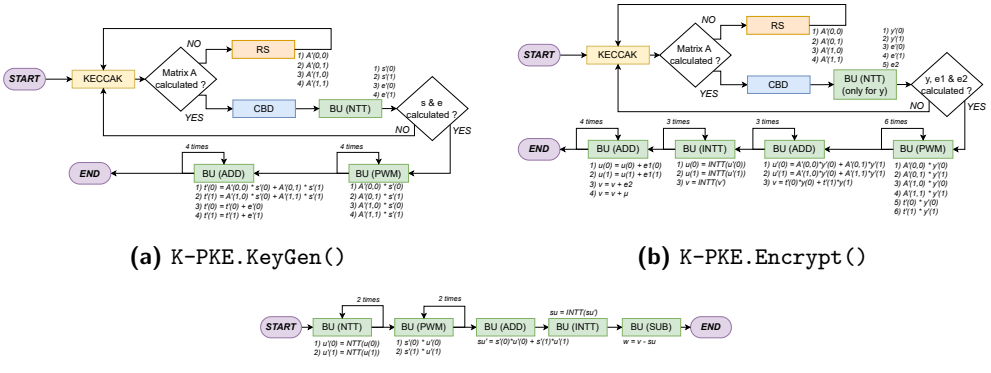
A similar flow applies to K-PKE.Encrypt (Figure 2b), following Algorithm 2. The calculation of the matrix $\hat{\mathbf{A}}$ and the secret vector $\hat{\mathbf{y}}$ proceeds identically to K-PKE.KeyGen. However, unlike key generation, the noise vectors $\mathbf{e_1}$ and e_2 are not transformed through NTT. The subsequent steps involve a sequence of PWM and ADD operations, followed by INTT and a final ADD to incorporate the noise. These steps yield $\mathbf{u}$ and v, which after encoding form the ciphertext. In contrast, the K-PKE.Decrypt operation (Figure 2c), as defined in Algorithm 3, is more direct and requires significantly fewer module interactions.

The same methodology can be extended to higher security levels (ML-KEM-768 and ML-KEM-1024) by scaling the number of times each operation is executed. For instance, in K-PKE.KeyGen for ML-KEM-768 the RS is invoked nine times, the CBD plus NTT six times, and the ADD nine times to produce the final vector $\hat{\mathbf{t}}$.

3.2 The Butterfly Unit

NTT implementations are well explored in the literature, with hardware and software counterparts [SSM+23]. Most existing solutions achieve high performance, requiring a large amount of hardware resources [TAMK25]. Some works implement only the NTT operation, leaving the INTT or PWM unaddressed [ZLL+21, KMSH24]. In contrast, our goal was twofold: first, to develop a more compact solution suitable for embedded systems, and second, to provide a complete implementation encompassing NTT, INTT, PWM, incorporating as well ADD, and SUB operations, critical and never addressed in the literature. To that end, we designed the Butterfly Unit (BU) as an arithmetic core to support the five different polynomial operations: NTT, INTT, PWM, ADD, and SUB.

To achieve this, we integrated multiple optimizations from the literature into a single butterfly unit. In particular, we incorporated previously proposed optimizations for



(a) K-PKE.KeyGen() **(b)** K-PKE.Encrypt()

(c) K-PKE.Decrypt()

Figure 2: HOPE-MLKEM modular methodology for K-PKE-512.

NTT/INTT following the methodologies of [LS19, ZLL+21], reduced the complexity of PWM using the Karatsuba algorithm as described in [XL21], employed Dadda tree structures for modular reduction following [YMÖS21], and leveraged ping-pong memories to enable parallelism as proposed in [ZLL+21]. To the best of our knowledge, this level of integration and implementation trade-offs has never been done for ML-KEM.

Figure 3 shows the design of the Butterfly Unit including these optimizations whose integration is depicted in the next subsections. Two dual port RAMs operate in a ping–pong scheme, with the control unit handling write operations and address generation. The unit is triggered by a `start` signal and configured via a `mode` input, which selects the operation to be executed. Details of the control logic are provided in the following section. In addition, a dedicated ROM_z stores all coefficients $\zeta^{\text{BitRev}_7(i)}$ in consecutive order as outlined in the following subsections.

Table 1 summarizes the comparison of our `BU` design with state-of-the-art architectures that target Artix-7 FPGAs. Our `BU` achieves 209 MHz, comparable to previous works while reducing LUT and FF usage by more than 50% relative to [KMSH24, XL21]. BRAM utilization is also lower, requiring only 2.5 blocks versus significantly higher counts in other designs [GLK22, KMSH24]. These optimizations yield the lowest reported ENS values, and the ATP results demonstrate a clear advantage for our architecture. The results highlight the effect of the integrated pipeline stages: while they slightly increase the total number of clock cycles compared to unpipelined designs, they deliver a substantial throughput improvement. Several recent works have proposed optimized datapaths for NTT, INTT, and PWM [XL21, YMÖS21, GLK22, KMSH24], but often omit other critical operations such as ADD and SUB, instead focusing on isolated components. For example, [KMSH24] addresses only the optimization of NTT.

In the following subsections, we present the resulting Arithmetic Unit architecture

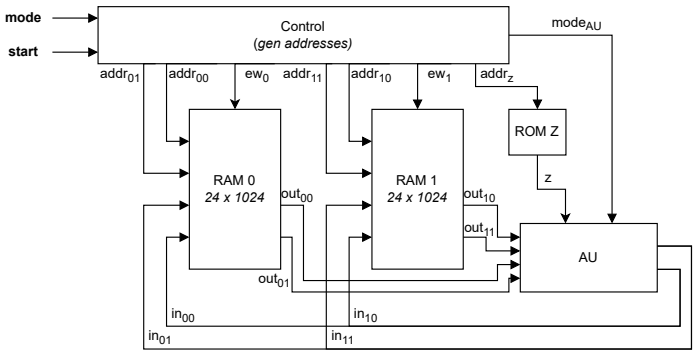


Figure 3: Top-level design of the Butterfly Unit

Table 1: Comparison of the Butterfly Unit with State-of-the-Art designs

	CCs [e]	Freq. (MHz)	LUTs	FFs	BRAMs	DSPs	ENS [c]	ATP [d,e]
[XL21] [b]	512 / 512 / 256 / - / -	161	3316	2225	6	4	3517.5	11186 / 11186 / 5593 / - / -
[YMÖS21]	232 / 233 / 167 / - / -	182	2543	792	9	4	3195.75	4073 / 4091 / 2932 / - / -
[BNAMK21]	474 / 602 / 1289 / - / -	115	737	290	4	6	1905.25	7852 / 9973 / 21355 / - / -
[GLK22]	448 / 448 / 128 / - / -	159	1549	792	16	4	4319.25	12169 / 12169 / 3477 / - / -
[KMSH24] [a]	254 / - / - / - / -	322	3821	2970	20	5	6860.25	5411 / - / - / - / -
This work	**517 / 517 / 137 / 70 / 70**	**209**	**1513**	**884**	**2.5**	**4**	**1710.25**	**4230 / 4230 / 1121 / 572 / 572**

[a] These results are for Zynq-UltraScale+.
[b] They provide results for server/client side. To a fair comparison they have been added together.
[c] *Equivalent Number of Slices:* $\text{ENS} = {}^{LUT}/_{4} + {}^{FF}/_{2} + DSP \cdot 100 + BRAM \cdot 196$ [LFK+19]
[d] *Area-Time-Product:* ATP = Latency (μs) · ENS
[e] [NTT / INTT / PWM / ADD / SUB]

within the Butterfly Unit. For each operation, we describe the applied design optimizations aimed at achieving an effective trade-off between area and performance.

3.2.1 NTT/INTT optimizations

Polynomial arithmetic in ML-KEM is performed in the ring $R_q = \mathbb{Z}_q[X]/(X^n + 1)$, knowing that $q = 3329$ and $n = 256$. In this case, $\mathbb{Z}_q$ contains n-th primitive roots of unity but no $2n$-th primitive roots. As a consequence, the polynomial $x^n + 1$ does not factor completely into linear terms; instead, for a primitive n-th root $\zeta \in \mathbb{Z}_q$ we obtain $x^n + 1 = \prod_{i=0}^{\frac{n}{2}-1}\left(x^2 - \zeta^{2i+1}\right)$, yielding a decomposition into $n/2$ irreducible quadratics. In that sense, for any polynomial $a(x) = \sum_{j=0}^{n-1} a_j x^j \in R_q$, an even/odd decomposition can divide it into two 128-term polynomials according to the index parity.

$$A_e(t) = \sum_{j=0}^{\frac{n}{2}-1} a_{2j}\, t^j, \qquad A_o(t) = \sum_{j=0}^{\frac{n}{2}-1} a_{2j+1}\, t^j, \qquad a(x) = A_e(x^2) + x\, A_o(x^2).$$

Letting $\theta_i = \zeta^{2i+1}$ for $0 \le i < \frac{n}{2}$, the NTT representation of a, $\hat{a}(x) = \hat{A}_e(x^2) + x\,\hat{A}_o(x^2)$, is given by the evaluations

$$\hat{A}_e[i] = \sum_{j=0}^{\frac{n}{2}-1} a_{2j}\, \theta_i^j \pmod q, \qquad \hat{A}_o[i] = \sum_{j=0}^{\frac{n}{2}-1} a_{2j+1}\, \theta_i^j \pmod q.$$

In the same way, for the inverse NTT, it is possible to reconstruct $a(x)$ from its NTT transforms $\left(\hat{A}_e[i], \hat{A}_o[i]\right)$. It proceeds by interpolating the values at the points θ_i, followed by recombination via

$$a_{2j} = n^{-1} \sum_{i=0}^{\frac{n}{2}-1} \hat{A}_e[i]\, \theta_i^{-j} \pmod q, \qquad a_{2j+1} = n^{-1} \sum_{i=0}^{\frac{n}{2}-1} \hat{A}_o[i]\, \theta_i^{-j} \pmod q,$$

for $0 \le j < \frac{n}{2}$, where n^{-1} denotes the modular inverse of n modulo q.

For efficient NTT and INTT operations, we adopted the decimation-in-time (DIT) strategy and the decimation-in-frequency (DIF) schedule, respectively, as initially presented in [LS19]. Both transforms operate independently on even and odd polynomial indexes. This configuration inherently means that the NTT processes a natural-order input to produce a bit-reversed output, while the INTT consumes a bit-reversed input to produce natural-order output. To reconcile these conventions and optimize performance, we implemented two key strategies: (i) Twiddle factors are supplied in bit reverse order and integrated directly with per-stage scaling, i.e., $\zeta^{\mathrm{BitRev}_7(i)}$, and (ii) the global factor n^{-1} in the INTT is distributed across stages as repeated multiplications by 2^{-1}. This approach significantly reduces the number of modular multiplications in $\mathbb{Z}_q$ and eliminates the need for explicit permutations. Algorithms 4 and 5 detail our optimized NTT and INTT implementations. Furthermore, we leverage the technique presented in [ZLL+21] to eliminate explicit divisions by 2 within the INTT algorithm, thus further reducing computational overhead.

$$\frac{x}{2} \bmod q = (x[0])\ ?\ (x \gg 1) + \left(\frac{q+1}{2}\right)\ :\ (x \gg 1) = (x \gg 1) + x[0] \cdot 1665 \tag{1}$$

3.2.2 Reducing complexity on PWM using Karatsuba algorithm

This specific reduction technique, drawing on methods detailed in [LS19, XL21], optimizes polynomial multiplication by recursively decomposing large operand multiplications into

smaller subproducts. This effectively reduces the number of multiplications required for this operation from five to four. Within the context of ML-KEM, the PWM is defined as:

$$\hat{c}_{2i} + x\,\hat{c}_{2i+1} = (\hat{a}_{2i} + x\,\hat{a}_{2i+1})(\hat{b}_{2i} + x\,\hat{b}_{2i+1}) \bmod (X^2 - \zeta^{2\mathrm{BitRev}_7(i)+1}). \tag{2}$$

Expanding this expression directly yields the following result as defined in [MLK24]

$$\hat{c}_{2i} = \hat{a}_{2i} \cdot \hat{b}_{2i} + \hat{a}_{2i+1} \cdot \hat{b}_{2i+1} \cdot \zeta^{2\mathrm{BitRev}_7(i)+1}\ ,\ \hat{c}_{2i+1} = \hat{a}_{2i} \cdot \hat{b}_{2i+1} + \hat{a}_{2i+1} \cdot \hat{b}_{2i}$$

In this formulation, a total of five multiplications in $\mathbb{Z}_q$ are required. Among these, the product $\hat{a}_{2i+1} \cdot \hat{b}_{2i+1} \cdot \zeta^{2\mathrm{BitRev}_7(i)+1}$ lies on the critical (longest) path, which cannot be shortened. However, the overall number of multiplications can be reduced using the Karatsuba algorithm, which reduces the number of n-bit multiplications from five to four. Thus, $\hat{c}_{2i} + \hat{c}_{2i+1}X$ is obtained as

$$\begin{aligned}
\hat{c}_{2i} &= \hat{a}_{2i} \cdot \hat{b}_{2i} + \hat{c}_{2i+1} \cdot \hat{b}_{2i+1} \cdot \zeta^{2\mathrm{BitRev}_7(i)+1}, \\
\hat{c}_{2i+1} &= (\hat{a}_{2i} + \hat{a}_{2i+1})(\hat{b}_{2i} + \hat{b}_{2i+1}) - (\hat{a}_{2i}\hat{b}_{2i} + \hat{a}_{2i+1}\hat{b}_{2i+1}) = \hat{a}_{2i} \cdot \hat{b}_{2i+1} + \hat{a}_{2i+1} \cdot \hat{b}_{2i}.
\end{aligned}$$

In our hardware design, this computation is segmented into two distinct states, aligning with the operational flow of the Butterfly Units:

- **PWM 1st stage:**

$$s_0 = \hat{a}_{2i} + \hat{a}_{2i+1}, \quad s_1 = \hat{b}_{2i} + \hat{b}_{2i+1}, \quad m_0 = \hat{a}_{2i}\hat{b}_{2i}, \quad m_1 = \hat{a}_{2i+1}\hat{b}_{2i+1}$$

- **PWM 2nd stage:**

$$s_2 = m_0 + m_1, \quad m_2 = s_0 \cdot s_1, \quad m_3 = m_1 \cdot \zeta^{2\mathrm{BitRev}_7(i)+1}$$

In that sense, it is possible to demonstrate that the following holds:

$$\begin{aligned}
\hat{c}_{2i} &= m_0 + m_3 &&= \hat{a}_{2i}\hat{b}_{2i} + \hat{a}_{2i+1}\hat{b}_{2i+1}\zeta^{2\mathrm{BitRev}_7(i)+1} \\
\hat{c}_{2i+1} &= m_2 - s_2 &&= \hat{a}_{2i}\hat{b}_{2i+1} + \hat{a}_{2i+1}\hat{b}_{2i}
\end{aligned}$$

Consequently, the PWM operation can be formalized as shown in Algorithm 6.

Algorithm 4 NTT Algorithm

Input $\mathbf{a} = (a[0], a[1], \ldots, a[n-1])$
Input $\zeta^{BitRev_7(i)}$
Output $\mathrm{NTT}(\mathbf{a}) = \hat{\mathbf{a}}$

$k \leftarrow 1$
for $i \leftarrow 1$ **to** 7 **do**
  $l \leftarrow 2^{7-i}$
  for $s \leftarrow 0$ **to** 256 **by** $j + l$ **do**
    $z \leftarrow \zeta^{BitRev_7(k)}$
    for $j \leftarrow s$ **to** $s + l$ **do**
      $t \leftarrow z \cdot a[j + l]$
      $u \leftarrow a[j]$
      $\hat{a}[j] = (u + t) \bmod q$
      $\hat{a}[j + l] = (u - t) \bmod q$
    end for
    $k \leftarrow k + 1$
  end for
end for

Algorithm 5 INTT Algorithm

Input $\hat{\mathbf{a}} = (\hat{a}[0], \hat{a}[1], \ldots, \hat{a}[n-1])$
Input $\zeta^{BitRev_7(i)}$
Output $\mathrm{INTT}(\hat{\mathbf{a}}) = \mathbf{a}$

$k \leftarrow 1$
for $i \leftarrow 7$ **to** 1 **do**
  $l \leftarrow 2^{7-i}$
  for $s \leftarrow 0$ **to** 256 **by** $j + l$ **do**
    $z \leftarrow \zeta^{BitRev_7(k)}$
    for $j \leftarrow s$ **to** $s + l$ **do**
      $t \leftarrow (\hat{a}[j] + \hat{a}[j + l]) \bmod q$
      $u \leftarrow (\hat{a}[j] - \hat{a}[j + l]) \cdot z \bmod q$
      $a[j] = t/2$
      $a[j + l] = u/2$
    end for
    $k \leftarrow k + 1$
  end for
end for

Algorithm 6 PWM Algorithm

Input $\hat{\mathbf{a}} = (\hat{a}[0], \hat{a}[1], \ldots, \hat{a}[n-1])$
Input $\hat{\mathbf{b}} = (\hat{b}[0], \hat{b}[1], \ldots, \hat{b}[n-1])$
Input $\zeta^{BitRev_7(i)}$
Output $\mathrm{PWM}(\hat{\mathbf{a}}, \hat{\mathbf{b}}) = \hat{\mathbf{c}}$

for $j \leftarrow 0$ **to** 128 **do**
  $s_0 \leftarrow \hat{a}[2j] + \hat{a}[2j + 1]$
  $s_1 \leftarrow \hat{b}[2j] + \hat{b}[2j + 1]$
  $m_0 \leftarrow \hat{a}[2j] \cdot \hat{b}[2j]$
  $m_1 \leftarrow \hat{a}[2j + 1] \cdot \hat{b}[2j + 1]$
  $s_2 \leftarrow m_0 + m_1$
  $m_2 \leftarrow s_0 \cdot s_1$
  $m_3 \leftarrow m_1 \cdot \zeta^{2\mathrm{BitRev}_7(j)+1}$
  $c[2j] \leftarrow m_0 + m_3$
  $c[2j + 1] \leftarrow m_2 - s_2$
end for

3.2.3 Modular multiplications: using Dadda Tree structures for modular reduction

One of the critical operations in NTT, INTT, and PWM is the multiplication of two 12-bit integers. In this work, such multiplications are performed using a DSP block with a pipeline stage, producing a 24-bit result that must subsequently be reduced modulo q. To efficiently handle this reduction, we apply an optimization technique based on Dadda tree structures, inspired by the approach of [YMÖS21] and originally used in [ZLL+21] for NewHope. The modular reduction leverages the specific structure of the ML-KEM modulus, $q = 3329 = 2^{12} - 2^9 - 2^8 + 1$. Given a 24-bit product i, repeated application of this decomposition enables the reduction [ZLL+21] :

$$r = 2^{12} i[23:12] + i[11:0] = (2^9 + 2^8 - 1) i[23:12] + i[11:0]) \tag{3}$$

The proposed modular reduction unit is implemented as shown in Figure 4. The 24-bit DSP output is first fed into the Dadda tree, which performs a sequence of additions and produces a partial result represented in carry–save form, with S denoting the sum and C the carry. The recombined result $(C + S)$ lies in the range $[-3264, 9271]$, which does not yet match the target interval $[0, q)$. To correct this, four candidate values are computed in parallel: $(C + S + q)$, $(C + S)$, $(C + S - q)$, and $(C + S - 2q)$. These candidates are stored in registers, introducing an additional pipeline stage. Finally, the correct representative in $[0, q)$ is selected as the reduced result, completing the operation in three clock cycles.

3.2.4 Ping-Pong Memories for parallelism

To maximize memory bandwidth and enable efficient parallelism, we adopt a ping–pong memory architecture, following the approach of [ZLL+21]. This dual port system alternates read and write operations across two memory banks, allowing the `BU` module to process one set of coefficients while simultaneously preparing the next. A further practical advantage of this design is scalability: the memory can be configured to support polynomials ranging from 128 to 1024 coefficients without increasing overall resource utilization.

In addition, since arithmetic operations naturally separate across even and odd indices, we employ a 24-bit datapath. The least significant 12 bits correspond to the even coefficients $(a[0], a[2], \ldots, a[254])$, while the most significant 12 bits correspond to the odd coefficients $(a[1], a[3], \ldots, a[255])$. This packing enables an entire ML-KEM polynomial to be stored

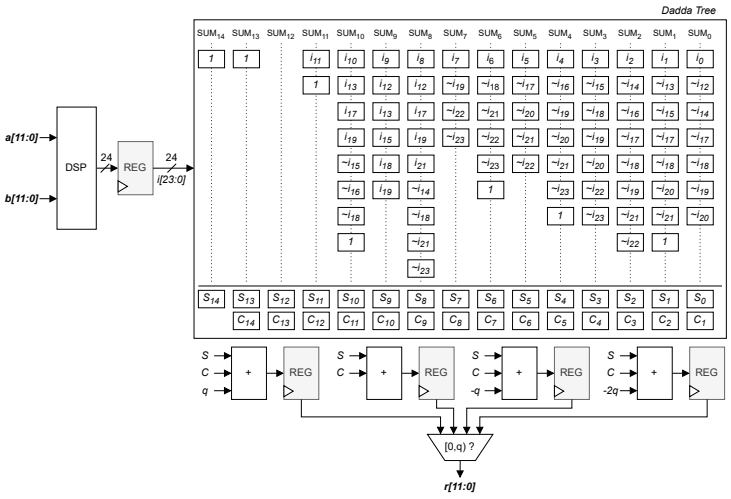
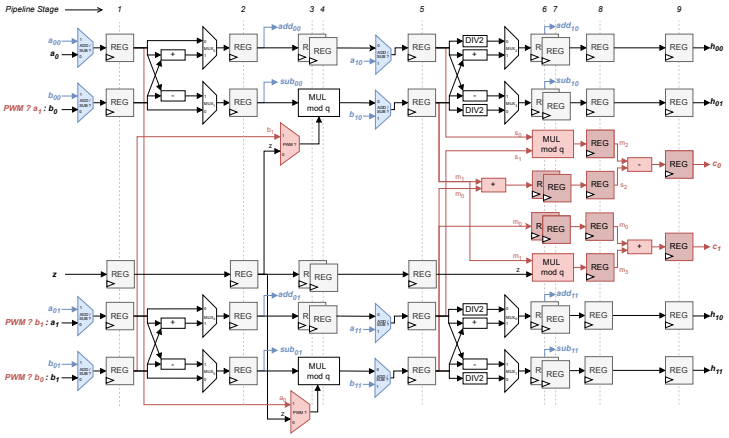


Figure 4: Modular multiplication module design based on Dadda Tree



Op.	MUX Index 7	6	5	4	3	2	1	0
NTT	1	1	0	0	1	1	0	0
INTT	0	0	1	1	0	0	1	1
PWM	0	0	0	1	0	0	0	1
ADD	0	1	0	1	0	1	0	1
SUB	1	0	1	0	1	0	1	0

Figure 5: Arithmetic unit of the Butterfly Unit. In gray the original datapath to perform NTT and INTT. In red included modifications to perform the PWM, while in blue the modifications to perform either ADD or SUB. The table indicates the Multiplexer control.

in just 128 memory locations. By restructuring the memory interface and reusing block RAM, the BU module achieves efficient storage and maintains the same resource footprint, while supporting the ability to process different polynomials at the same time.

3.2.5 Final design of the Arithmetic Unit of the Butterfly Unit

The proposed arithmetic unit for the BU is depicted in Figure 5, which supports NTT, INTT, and PWM (Algorithms 4, 5, and 6). The unit is organized as a nine-stage pipeline, with its datapath controlled by configurable multiplexers. While our design draws inspiration from the configurable multiplexer approach presented in [XL21], we significantly expand on this concept. Our enhancements lead to a more compact and higher performance architecture, capable of simultaneously supporting all required arithmetic operations. In contrast to previous work [YMÖS21, XL21], which primarily emphasizes pipelining and register placement, our design also integrates both ADD and SUB operations. This is enabled by partitioning the arithmetic unit into two independent sections that exploit dual-port RAM access, allowing four values to be processed in parallel. As a result, the latency of addition and subtraction is effectively halved.

3.3 Keccak Unit

Following the description of the required modules, we now turn to the `Keccak` module. While the Keccak function is essential for ML-KEM operations, the detailed design and optimization of the `Keccak` hardware unit are beyond the scope of this work. To that end, the `Keccak` unit is based on the design from [Saa24], which we modified to expose a minimal control interface consisting of the signals `load`, `start`, and `read`. A separate control module selects the required functionality among SHA3-512, SHA3-256, SHAKE128, and SHAKE256, ensuring compliance with the respective padding rules while adapting the input formatting to each algorithm. The unit requires 1 cycle to load a block, 24 cycles to perform the permutation, and 1 cycle to read the result.

3.4 CBD and Rejection Sampler

The CBD and RS modules implement the SamplePolyCBD(·) and SampleNTT(·) functions, respectively. Both employ a shift register to buffer the Keccak output. In the CBD module, the distribution is determined by the parameter η. As two 24-bit words can be written per cycle, four coefficients are processed in parallel, resulting in 64 cycles to generate a complete polynomial. The RS module follows a similar design, buffering the Keccak output in a shift register and producing four 12-bit samples per cycle. Unlike the CBD case, this procedure is inherently non-constant time [MLK24], since only coefficients below q are accepted and sampling continues until a complete polynomial is obtained. To support this, the RS module integrates a dedicated BRAM to store the generated polynomial, which is later used independently in subsequent stages of the algorithm.

3.5 Control Logic

One of the key innovations over previous ML-KEM hardware implementations is that our design is **fully programmable**. This flexibility enables a wide range of possible implementations to be explored. Central to this capability is the control logic, which coordinates both the Keccak+CBD+RS datapath and the Butterfly Units. For the Keccak+CBD+RS datapath, control is realized through a microprogrammed FSM. The instruction set is stored in ROM, and the FSM selects the entry point based on the operation mode, i.e., key generation, encapsulation or decapsulation with $K \in 2, 3, 4$ corresponding to ML-KEM-512, ML-KEM-768 and ML-KEM-1024 respectively. Once initialized, the FSM sequentially fetches and executes 16-bit control words, whose format is summarized in Table 2. The semantics of the instruction fields vary depending on the instruction mode:

- **CBD mode** (01-00 = 00): The upper four bits (15-12) specify the storage location of the resulting polynomial, which is particularly useful in multi-BU configurations. The memory offset is also controlled by bit 07. The counter *cnt*, defined in Algorithms 1 and 2 as the variable associated with σ in SamplePolyCBD(·), is encoded in bits 11-08, while bits 06-05 configure η. Bit 04 selects the seed (ρ for SampleNTT(·), or σ/r for SamplePolyCBD(·)). Finally, the Keccak counter field specifies the number of full Keccak executions, serving primarily as auxiliary information.
- **RS mode** (01-00 = 01 or 10): Bits 15-12 and 11-08 encode the indices i and j, as defined in Algorithms 1 and 2 for SampleNTT(·). The offset and η fields are unused. Seed selection is fixed to ρ, while the Keccak counter retains the same role as in CBD mode.
- **END mode** (01-00 = 11): Terminates program execution.

Table 2: Control instruction for Keccak+CBD+RS modules

control [15-00]						
15 - 12	**11 - 08**	**07**	**06 - 05**	**04**	**03 - 02**	**01 - 00**
{BU_sel, off }	*cnt*	extra_off	η (10 / 11)	seed_sel	keccak_cnt	**00 (CBD)**
i	j	-	-	ρ (0), σ | r (1)		**01 / 10 (RS)**
-	-	-	-	-	-	**11 (END)**

The microprogrammed FSM is explicitly designed to enforce constant-time behavior, preventing timing-based leakage, specially on sensitive data. For public data whose operations follow non-deterministic flows (i.e., SampleNTT), we enforced a upper bound on the number of loop iterations, consistent with the specification in FIPS 203 [MLK24].

The BU control program is controlled by a similar FSM, sharing the same core control mechanism despite differences in instruction sets. This instruction set is tailored to the BU's architecture, specifically its two dual-port RAMs operating in a ping-pong configuration. To facilitate multi-BU systems, where control logic must coordinate operations across units

by accessing various RAMs, we introduce the Data Management Unit (DMU). The DMU is a fundamental component responsible for directing data flow and managing RAM I/O in these multi-BU setups. Table 3 details the instruction format for the BU control program. Each instruction is 20 bits wide with distinct fields, enabling flexible data routing and multi-unit support. The upper three fields (19-16, 15-12, and 11-08) define memory offsets for the external RAM (RAM from other BU), RAM-1, and RAM-0, respectively. These offsets allow precise addressing of coefficients within the dual-port RAMs, which operate in ping-pong mode. The 07-04 field specifies the DMU control, determining whether the BU is used for NTT/INTT operations, PWM, or multi-BU coordination. When multiple BUs are instantiated, the DMU assigns operations to a specific BU, managing shared resources such as RAMs. Bit 03 defines the read/write control: in mode 0, RAM-1 is read and RAM-0 written, while in mode 1 the roles are reversed. Finally, the 02-01 field encodes the BU operation mode, which includes NTT (001), INTT (010), PWM (011), ADD (100), SUB (101), and a dedicated wait mode (110) used to synchronize on a predefined number of clock cycles.

Table 3: Control instruction for Butterfly Units

control [19-00]					
19 - 16	**15 - 12**	**11 - 08**	**07 - 04**	**03**	**02 - 01**
ext RAM offset	**RAM-1 offset**	**RAM-0 offset**	**DMU control**	**R/W control**	**BU mode**
0: 0	0: 0	0: 0	1: NTT/INTT	0 (RAM-1 Read, RAM-0 Write)	**001 (NTT)**
1: 128	1: 128	1: 128	2: PWM RS0	1 (RAM-1 Write, RAM-0 Read)	**010 (INTT)**
2: 256	2: 256	2: 256	3: PWM RS1		**011 (PWM)**
3: 384	3: 384	3: 384	8: PWM BU		**100 (ADD)**
4: 512	4: 512	4: 512	4: BU 1		**101 (SUB)**
5: 640	5: 640	5: 640	5: BU 2		
6: 768	6: 768	6: 768	6: BU 3		
7: 896	7: 896	7: 896	7: BU 4		
CCs [12 bits]			-	-	**110 (WAIT CCs)**

3.6 Scheduling strategy

The proposed methodology efficiently uses hardware components to support a wide range of implementation choices. Although operations are structured sequentially, the key insight lies in exploiting intermediate results independence. This enables opportunities for partial parallelism. A carefully designed scheduling strategy overlaps independent operations, improving hardware utilization, and significantly reducing overall latency.

Figure 6 illustrates a possible scheduling derived from the K-PKE.KeyGen for ML-KEM-512, as described in Figure 2a. In this specific case, the operation NTT can be initiated immediately after the calculation of the vectors **s** and **e** within the CBD module. In this implementation, four BUs ($N_{BU} = 4$) are considered. While NTT is in progress, the RS module can already begin to calculate the matrix $\hat{\mathbf{A}}$. This analysis assumes a single Keccak unit where the CBD and RS modules could proceed fully independently. Once the

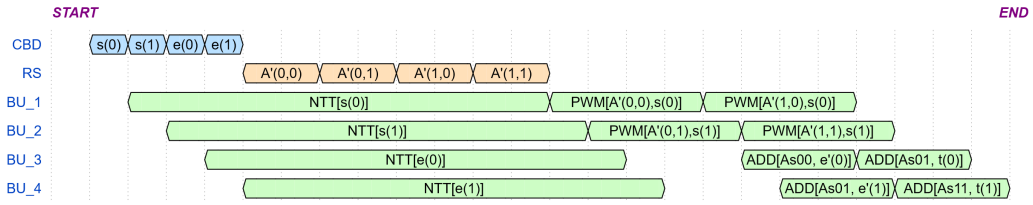
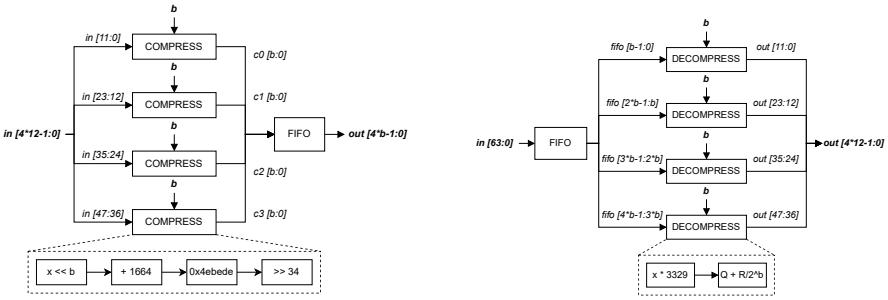


Figure 6: Example of scheduling strategy for K-PKE.KeyGen for ML-KEM-512



(a) `Encoding + Compression` module **(b)** `Decoding + Decompression` module

Figure 7: Encoder and Decoder modules

calculations of $\hat{\mathbf{A}}$ and NTT are complete, the operation PWM is started, followed by ADD as soon as its input becomes available.

In contrast to previous approaches [XL21, NKK+23, NKLO25], where the number and configuration of hardware modules are fixed at design time, our configurable framework allows modules to operate completely independently. This architectural decoupling yields an essentially unbounded design space for personalization and optimization. In this sense, our work provides a fully programmable framework for implementing hardware ML-KEM cores. Although in Figure 6 we use four `BU`s, a designer can select the appropriate number of units according to their needs, ranging from less `BU`s for more compact architectures to substantially more units to achieve high-performance implementations. This also aligns with the principles required to achieve high-order security, where independent shares can operate autonomously as we exploit in Section 4.

3.7 Encoding and Decoding modules

For the `Encoding + Compression` and `Decoding + Decompression` modules, our design approach incorporates the countermeasures proposed in [JD23] as illustrated in Figure 7. This previous work suggests that message recovery attacks would be more challenging if multiple polynomial coefficients were converted into their corresponding bytes in parallel during decoding, rather than individually, as is commonly done in previous software and hardware implementations.

The encoding process is optimized for hardware by transforming all divisions into multiplications, improving the proposal of [KJSL24] in several pipeline stages. Specifically, the first pipeline stage performs a left shift $xs = x \cdot 2^b = x \ll b$, where b can be d_u or d_v. The second stage replaces a division with a multiplication and shift-based reduction:

$$o = \frac{xs}{3329} = \left(\frac{xs \cdot 2^{34}}{3329}\right) >> 34 = (xs \cdot \text{0x4edebe}) >> 34$$

We also add 1664 to ensure that the result is always in the range $[0, q)$. In this context, we introduced an additional optimization that replaces constant multiplications with equivalent shift-and-add operations.

The decompression process is also pipelined for efficiency, avoiding multiplications/divisions. The first step multiplies by the constant q, expressed as $xm = x \cdot 3329 = 2^b \cdot Q + R$, where Q and R denote the quotient and remainder, respectively. Since this result must be divided by 2^b, the decompressed output can be expressed as

$$o = Q + \frac{R}{2^b}\ ,\ Q = xm \gg b\ ,\ R = xm \wedge (2^b - 1)$$

4 Side-Channel Countermeasures

4.1 Sensitivity analysis

In order to propose countermeasures, the initial step involved the identification of possible attack targets. Unlike [ZPM+23], we extend the analysis to other parts of the algorithm beyond `K-PKE.Decrypt()`. In the `ML-KEM.KeyGen()`, Algorithm 1, there are several secret variables whose disclosure, even partial, enables recovery of the decapsulation key. Concretely, these include: the seed d used to expand both encapsulation (public) and decapsulation (secret) key material, the value σ used to derive the decapsulation key, and the secret vector $\mathbf{s}$ and the error vector $\mathbf{e}$. Leakage of any of these variables compromises security. For example, knowledge of $\mathbf{e}$ often enables efficient recovery of $\mathbf{s}$ since $\hat{\mathbf{A}}$ is public. More generally, recovering any of the above values provides an adversary with the means to reconstruct the decapsulation key and thus break the scheme. Consequently, protection cannot be limited to the seed alone: every intermediate value produced during `ML-KEM.KeyGen()` must be treated as sensitive. This requirement is not limited to key material alone, but extends to the operations that process secret data, i.e. NTT, INTT, PWM, or encoding functions, all of which manipulate secret polynomials and, therefore, could expose leakable intermediate states. Side channels that reveal intermediate coefficients, pointwise products, or partial sums can provide sufficient information to reconstruct $\mathbf{s}$, $\mathbf{e}$, σ, or the seed d.

Similar analysis can be carried out for `ML-KEM.Encaps()`, Algorithm 2. In that case, the message m together with the randomness derived from it, i.e., K and r, must be regarded as sensitive. Leakage from operations involving m may directly compromise confidentiality. In this context, instead of σ, the critical seed is r, used in the $\mathsf{SamplePolyCBD}_\eta(\cdot)$ function. In this part of the scheme, the most sensitive part is the computation of c_2, which depends on the vector v. Recovering partial information about v can lead to the reconstruction of the vector μ, which is directly related to the message m.

Regarding the decapsulation, `ML-KEM.Decaps()`, Algorithm 3, two parts require special attention. First, during `K-PKE.Decrypt()`, the sensitive values derived from $\mathsf{dk}_{\mathsf{PKE}}$ are used to generate the secret vector $\hat{\mathbf{s}}$. Any leakage of these values would immediately enable the recovery of m', which would break the scheme. Second, the decapsulation procedure reuses `K-PKE.Encrypt()` for the re-encryption process where the same outlined conclusions obtained above about the sensitivity of m, r, and related intermediates apply.

In summary, the threat model must regard every intermediate variable and each arithmetic or transform operation involving secret data as sensitive. Side-channel protections must therefore secure not only long-term secrets but also ephemeral intermediate values and the datapath logic that processes them.

4.2 Side-Channel countermeasures and attacks in literature

Efforts toward secure implementations of Kyber/ML-KEM began with the software-oriented work of [BGR+21]. Their approach introduced a bit-sliced binary search method and explored the trade-off between using uncompressed masked polynomials and compressed public polynomials, thereby avoiding the need to mask the ciphertext compression. This line of research was later refined and optimized for the ARM Cortex-M4 platform in [HKL+22]. Building on the idea of bitslicing, [BC22] proposed masked implementations for lattice-based cryptography that included dedicated masked arithmetic-to-boolean (A2B) and boolean-to-arithmetic (B2A) conversion modules, specifically tailored to the compress and decompress operations in Ring-LWE schemes. However, the latter design was shown to be vulnerable to message-recovery attacks [WBD23, JNWD24].

For masked hardware implementations, the first attempt to incorporate countermeasures was made in [JGCS21], later updated in [JGCS24], which explored the use of random delays and instruction shifts. Later, [XL21] introduced a constant-time implementation,

but this design was shown to be vulnerable to the sliced multi-bit error injection attack of [JWN+22]. The effectiveness of this attack highlights a key distinction: while software implementations execute instructions sequentially, slicing becomes essential in hardware realizations. Subsequent work in [KNAH22] combined masking with hiding techniques, but was again broken in [JD23] due to the same vulnerability. More recently, [MJB+24] proposed the use of random clocks and duplication to reduce energy correlation at the cost of significant resource overhead. In parallel, [XWT25] introduced an optimized shuffling-based architecture presenting results for the decryption process.

4.3 Proposed countermeasures

Previous work in the literature has often focused on protecting specific, isolated components, particularly in hardware implementations. However, our approach promotes a comprehensive protection strategy that covers all stages of the ML-KEM operation. This comprehensive perspective aims to mitigate a wider range of side-channel attacks that could exploit intermediate values as previously identified. To that end, our proposal leverages and expands the design methodology, used for the hardware design outlined in the previous section, to incorporate robust countermeasures for timing and power attacks. Specifically, our proposed countermeasures include the following:

- **Protection of linear operations via masking:** Masking of linear operations performed by the `BU` (i.e., NTT, INTT, PWM, ADD, SUB) is straightforward. Masks can be generated and applied independently at each stage, with recovery performed as needed. To that end, we leverage the scheduling strategy detailed in the previous section to operate with independent shares in each `BU`, where the program control can be adapted to process different shares in parallel. This enables a multi-`BU` approach thereby facilitating scalability to higher security orders with limited performance impact. For example, with $N_{\texttt{BU}}=2$, two independent shares can be processed yielding first-order protection. Extending this to $N_{\texttt{BU}}=4$ allows for four independent shares, thereby achieving third-order protection. Another option would be two independent shares using $N_{\texttt{BU}}=4$, achieving a higher performance with a first-order protection.
- **Decorrelation of the shares:** When applying masking countermeasures in hardware implementations, it is recommended to avoid processing multiple shares in parallel within the same clock cycle. Instead, temporal non-completeness should be enforced by scheduling shares sequentially rather than simultaneously [DEM18]. In our design, each `BU` operates independently and never within the same cycle. A configurable constant delay (at least one cycle) separates the initiation of each `BU`. For example, with $N_{\texttt{BU}} = 2$ and a delay of 10 cycles, the second `BU` begins its operation 10 cycles after the first. To further increase unpredictability, we randomize the order in which `BU`s are activated. For example, with $N_{\texttt{BU}} = 4$, there are $4! = 24$ possible initiation sequences. This randomization prevents adversaries from aligning traces across shares, unless the randomness source itself is compromised.
- **Masked `CBD` sampling:** The `CBD` module converts secret values from `Keccak`'s binary output into an arithmetic representation in $\mathbb{Z}_q$. Integrating this module within a masked implementation requires a robust binary-to-arithmetic (B2A) conversion process. However, efficient and compact B2A conversions suitable for high-order hardware masking are still under study. A recently presented gadget [SH24] offers protection up to second-order. Its implementation achieves the conversion in four cycles at a cost of 1,395 LUTs, 1,676 FFs, and 13 DSPs per 32-bit word. Scaling this approach to a high-order scheme, using a lower bound of 34 instances of the B2A gadget to mask the `Keccak` output, would require an area overhead of 47,430 LUTs, 56,984 FFs, and 416 DSPs per share if the `Keccak` output must be parsed with minimal latency. Such an area and performance penalty is prohibitive for our design. To address this efficiency challenge, we employ a method based on Secure

Double-Rate Registers (SDRR) [BBM+18]. To implement this, the `CBD` operation is carried out over twice the usual number of clock cycles. During one of these cycles, intermediate values are randomized using fresh randomness, ensuring that the power consumption is decorrelated from the sensitive data being processed.

- **Masked Keccak implementations:** One critical component of the ML-KEM scheme is the Keccak operation, which can be a potential source of side-channel leakage. Although this is not the main focus of our work, we sought to investigate complete countermeasures in order to provide a fully secure solution. We therefore explored two well-known masking approaches for Keccak: Threshold Implementations (TI) [NRR06] and Domain-Oriented Masking (DOM) [GMK16]. In TI, every nonlinear operation is computed on shares using dedicated TI gadgets, offering provable first-order resistance against leakage. In contrast, DOM applies Boolean masking across the state and secures each operation with masked AND gadgets. For our work, we implemented a 3-share TI based on the design presented in [Saa24]. However, as prior studies have raised concerns about the actual security guarantees at the share level [Rep15, MS16], we limited TI to first-order protection. To provide higher-order protection, we additionally designed a new DOM implementation with 4 shares, developed from scratch following [ZSS+21].

4.4 Masked Dataflow and Composability

Building on the top-level architecture shown in Figure 1, Figure 8 explicitly details the domain separation and the lifecycle of the masked shares. The architecture strictly partitions the design into a public domain (depicted in blue) and a masked domain (depicted in red). While the diagram illustrates a configuration with two shares for clarity, the framework is generic and can be extended to support higher masking orders. Randomness is supplied by a dedicated `RNG` module. In this specific implementation, we utilize an LFSR to seed an external `Keccak` instance for randomness expansion. However, the modular interface allows this PRNG to be seamlessly replaced by another one or a physical TRNG.

Share Lifecycle and Randomness. As illustrated in Figure 8, the lifecycle of sensitive data is managed from generation to consumption as follows:

- **Input Boolean Masking:** Sensitive inputs requiring hashing (e.g., the seed d during Key Generation) enter through the *Masked Input* interface of the Keccak Stage. Fresh randomness from the `RNG` is immediately applied to generate Boolean shares before the data is processed by the masked `Keccak` core.
- **Share Generation in `CBD` module (SDRR):** The transition from the Boolean domain to the Arithmetic domain is managed by the `CBD` unit. Here, the `RNG` injects fresh randomness to drive the SDRR logic. This mechanism converts Boolean shares into arithmetic shares (s_0, s_1) while randomizing intermediate values, ensuring that the resulting shares are stored in independent banks within the RAM module.
- **Arithmetic Share Splitting:** For sensitive data entering the arithmetic domain directly (e.g., m during Encapsulation), the `Decoder+Decompress` utilizes the `RNG` to split the input into arithmetic shares (s_0, s_1) before passing them to the `DMU`.

Propagation and Independence. Once inside the arithmetic domain, the shares propagate distinct datapaths. The `DMU` controls this flow, routing specific shares to their designated `BU`s and `RAM` banks. Crucially, as depicted by the double red lines in Figure 8, shares' operations are executed independently and in sequential clock cycles. This temporal separation ensures that the combinatorial logic never processes multiple shares simultaneously, thereby preventing leakage related to the Hamming distance of the combined secret as it was aforementioned. Each `BU` has access to the public matrix $\hat{A}$, while the `DMU` handles replication and regulates access to specific elements.

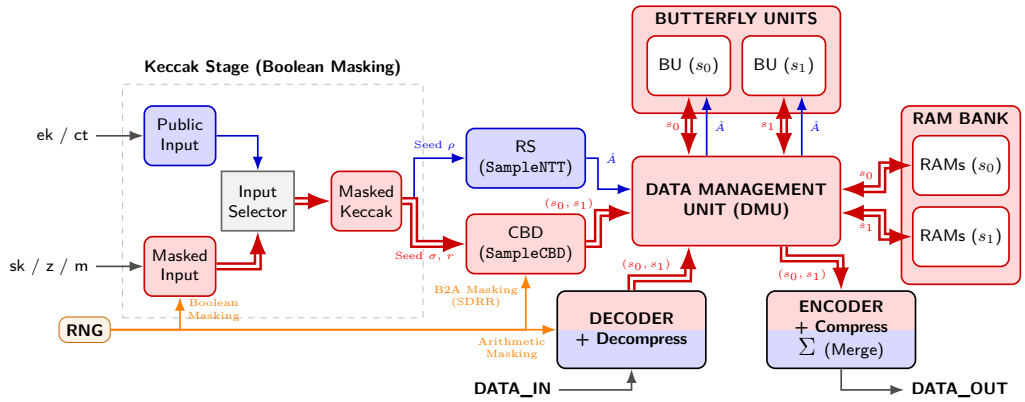


Figure 8: Detailed dataflow of the masked architecture. Blue regions indicate public domains; Red regions indicate sensitive domains. The control logic has been removed to ease the understanding the dataflow.

Recombination and Domain Boundaries. The boundaries between the masked and public domains are explicitly enforced at module interfaces, visualized in Figure 8 via split-color backgrounds:

- **Recombination (`Encoder`):** Share recombination is strictly confined to the `Encoder + Compress` stage (marked with the symbol Σ). The adder within this module sums the shares ($c = s_0 + s_1 \bmod q$) exclusively when generating public outputs, such as the ciphertext or the shared secret.
- **Isolation:** Public operations, such as the generation of the matrix $\hat{\mathbf{A}}$ in the RS module, operate entirely on the blue path using the public seed (ρ), maintaining physical and logical isolation from the secret data path.

Composability. The security of the composed countermeasures relies on the statistical independence of the randomness injected at each transition point. While the `Keccak` unit employs TI or DOM for Boolean operations, its output is not directly mapped to arithmetic values. Instead, it is re-randomized via the SDRR mechanism within the `CBD` unit. The global `RNG` guarantees that the masks utilized for Boolean masking, SDRR re-randomization, and arithmetic splitting remain statistically independent.

5 Results

In this section, we first present a performance comparison with the state of the art and then a security evaluation. To ensure transparency and reproducibility, the accompanying repository [CRNTCA] provides all the source code needed to reproduce our analysis, along with tools for simulation, synthesis, and security evaluation. The evaluated implementation corresponds to the `TOP_MLKEM` design included in the repository. For convenience, we also provide an AXI interface and the corresponding drivers, enabling integration with an external processor. However, note that external communication interfaces are excluded from our comparison, as they do not affect the core cryptographic performance. We also exclude the modules based on Keccak and the LFSR used for random share generation in masked implementations.

5.1 Performance Analysis

In this subsection, we review prior work on software and hardware implementations of ML-KEM presented in the literature, and we introduce the ASIC implementation flow.

Comparison with ARM Cortex-M4 software implementations. The most widely used repository providing optimized Kyber/ML-KEM software implementations for ARM Cortex M4 microcontrollers is `pqm4` [KRSS19]. These implementations combine hand-optimized assembly routines for the NTT with targeted modifications of the reference C code to reduce stack usage and overall memory footprint. However, even with such platform-specific optimizations, public-key operations on a Cortex M4 remain costly: key generation, encapsulation, and decapsulation typically require on the order of 10^6 cycles.

Comparison with HW/SW RISC-V based implementations. Most prior work has focused on ISA extensions for RISC-V [NDMZ+21, ZYHK24]. In particular, [ZYHK24] presents optimizations for Keccak and Kyber, achieving reductions of up to 500k cycles for Kyber-768 on an RV32IMC core and up to 230k cycles on an RV64IMC core. The work of [FSS20] introduces RISQ-V, a RISC-V design with tightly coupled hardware accelerators that enable efficient reuse of processor resources and reduce memory accesses. This approach reduces the execution time of Kyber-768 by up to 273k cycles, with a reported resource utilization of 24,300 LUTs, 10,800 FFs, 32 BRAMs, and 21 DSPs.

Comparison with full hardware implementations. Table 4 shows the comparison in terms of performance and area with recent work in the state of the art, where each row corresponds to a different security level (1, 3, and 5). For comparison with previous hardware implementations, we evaluated our ML-KEM module featuring different numbers of butterfly units ($N_{\mathtt{BU}} = 1, 2, 4$), on a Xilinx Artix-7 (XC7A200T-3) FPGA, using Vivado 2023.1 with default synthesis settings. Among existing works, only [XL21] provides open-source code (Pub.), which facilitates reproducibility. Regarding configurability (Conf.), defined as the ability to dynamically switch between security levels at runtime, only [KJSL24] explicitly supports this. Several works report separate implementations for client and server roles [XL21, NKK+23, NKLO25] or for each distinct security level [BNAMK21, DMG23, NKK+23, NKLO25]. For $N_{\mathtt{BU}} = 1$, our most compact implementation achieves resource utilization (ENS) comparable to that reported in [XL21, DMG23], while offering both configurability and protection against timing attacks. For $N_{\mathtt{BU}} = 2, 4$ resource utilization naturally increases relative to the $N_{\mathtt{BU}} = 1$ configuration. These configurations achieve the same Area-Time Product (ATP) as existing configurable designs while still maintaining protection against timing attacks. In this respect, our implementations surpass other works such as [XL21, BNAMK21]. A direct comparison with works like [DMG23, NKK+23, NKLO25] proves to be more challenging due to their non-configurable nature. While their reported ATP might appear superior in isolation, for a fair evaluation, considering a single configuration for these works and aggregate their ENS across all security levels, their effective ATP would exceed the results presented in this work.

Comparison with other side-channel protected implementations. This work represents the first in the literature to offer a fully configurable open-source implementation that provides first-order and high-order protection for ML-KEM. Since the `Keccak-TI3` module has been included for first order (16,293 LUTs and 4,814 FFs) and the `Keccak-DOM` for high order protection (27,035 LUTs and 6,414 FFs), there is an increase in the resources used. A direct comparison with related works is challenging. In particular, the work presented in [KNAH22] does not include key generation, reporting an ENS that exceeds 100k, significantly higher than our implementations, with an ATP substantially larger, featuring an encapsulation process of 88k cycles at 100 MHz. Similarly, [MJB+24, XWT25] exclusively presents results for decryption. [JGCS24] is the only work that provides comprehensive results; however, its design is configurable in the sense that specific countermeasures can be enabled / disabled, and there is no discussion of high-order countermeasures. Furthermore, it focuses on providing protection for specific components rather than a holistic solution.

Table 4: Comparison of ML-KEM Hardware Implementations

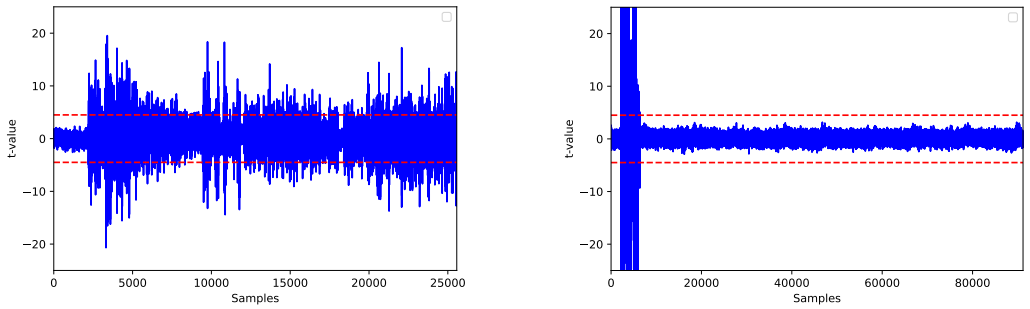
Ref.	Clock Cycles (k)			Latency (μs)			Freq.	LUTs	FFs	BRAMs	DSPs	ENS [b]	ATP [c]	Conf.	Pub.	SCA [d]
	KG	EN	DE	KG	EN	DE	(MHz)									
Unprotected implementations																
	4.0	7.0	10.0	34.8	60.9	87.0	115	18000	5000	15	6	10540	6.05			
[BNAMK21] [e]	7.0	10.0	14.0	60.9	87.0	121.7	115	16000	6000	16	9	11036	8.93	✗	✗	○
	10.0	14.0	18.0	87.0	121.7	156.0	112	16000	6000	17	12	11532	12.07			
	2.1	3.3	4.5	9.7	14.8	20.3	220	9347	8186	6	4	8006	1.29			
[DMG23] [e]	2.7	3.9	5.0	12.3	17.7	22.9	220	10434	9473	8.5	6	9611	1.52	✗	✗	○
	3.6	4.8	6.0	16.3	21.8	27.1	220	11527	10767	10.5	8	11123	1.87			
	1.1	1.5	2.1	5.3	7.2	10.1	208	30146	26371	0	4	21122	1.45			
[NKK+23] [a,e]	1.7	2.4	3.0	8.2	11.5	14.4	208	31630	26204	0	4	21410	2.20	✗	✗	○
	2.7	3.4	4.1	13.0	16.3	19.7	208	33493	26455	0	4	22001	3.16			
	1.8	1.9	3.2	8.6	9.1	15.2							0.68			
[KJSL24]	2.6	2.7	4.1	12.4	12.9	21.0	210	25757	18629	14	21	20598	0.95	✓	✗	○
	3.4	3.5	5.8	16.2	16.7	27.6							1.25			
	1.8	2.7	3.7	6.7	9.9	13.7	270	19028	15372	8	0	14011	1.33			
[NKLO25] [a]	3.3	4.2	5.4	12.3	15.5	20.0	270	19962	15563	8	0	14340	2.10	✗	✗	○
	5.1	6.0	7.6	19.3	22.6	28.7	263	21986	16678	8	0	15404	3.09			
Protected implementations																
	3.8	5.1	6.7	23.4	30.5	41.3							0.90			
[XL21] [a,e]	6.3	7.9	10.0	39.2	47.6	62.3	161	14197	8625	6	4	9438	1.41	✗	✓	◑
	9.4	11.3	13.9	58.2	67.9	86.2							2.00			
[JGCS24] [e,f]	13.8	17.2	22.1	53.6	66.7	85.6	258	7151	3730	5.5	2	4931	3.05			
Partially	26.6	30.9	37.8	103	119	146	258	7151	3730	5.5	2	4931	5.44	✗	✗	●
First-Order	43.8	48.8	57.2	170	189	224	258	7151	3730	5.5	2	4931	8.62			
This work	**3.7**	**5.4**	**9.0**	**20.6**	**30.0**	**50.0**							**1.01**			
($N_{\mathtt{BU}} = 1$)	**6.3**	**9.0**	**13.5**	**35.0**	**50.0**	**75.0**	**180**	**14436**	**10127**	**5**	**4**	**10053**	**1.61**	✓	✓	◑
	10.1	**14.5**	**19.8**	**56.1**	**80.6**	**110.0**							**2.48**			
This work	**2.1**	**3.2**	**4.4**	**11.7**	**17.7**	**24.4**							**0.64**			
($N_{\mathtt{BU}} = 2$)	**4.3**	**5.8**	**7.8**	**23.9**	**32.2**	**43.3**	**180**	**16299**	**11150**	**7.5**	**8**	**11919**	**1.18**	✓	✓	◑
	5.2	**7.2**	**9.9**	**28.9**	**40.0**	**55.0**							**1.48**			
This work	**1.3**	**2.7**	**3.9**	**7.2**	**15.0**	**21.7**							**0.72**			
($N_{\mathtt{BU}} = 4$)	**2.5**	**3.2**	**4.6**	**13.9**	**17.8**	**25.6**	**180**	**21259**	**13464**	**14.5**	**16**	**16489**	**0.94**	✓	✓	◑
	3.6	**4.6**	**5.7**	**20.0**	**25.6**	**31.7**							**1.27**			
This work	**4.7**	**6.5**	**9.0**	**27.6**	**38.2**	**52.9**							**3.07**			
First-Order	**7.6**	**10.3**	**13.7**	**44.7**	**60.6**	**80.6**	**175**	**43473**	**27101**	**7.5**	**8**	**26689**	**4.82**	✓	✓	●
	11.6	**16.2**	**20.3**	**68.2**	**95.3**	**119.4**							**7.33**			
This work	**5.8**	**7.6**	**11.2**	**33.1**	**43.4**	**64.0**							**5.61**			
High-Order	**8.7**	**11.4**	**15.9**	**49.7**	**65.1**	**90.9**	**175**	**67757**	**37057**	**14.5**	**16**	**39909**	**8.21**	✓	✓	●
	12.7	**17.3**	**22.5**	**72.6**	**98.9**	**128.6**							**11.9**			

[a] They provide separate results for the server side and the client side. In order to facilitate a fair comparison, the resources have been consolidated.
[b] *Equivalent Number of Slices:* ENS = $^{LUT}/_4 + {}^{FF}/_2 + DSP \cdot 100 + BRAM \cdot 196$ [LFK+19]
[c] *Area-Time-Product:* ATP = Total Latency $(s) \cdot$ ENS. To ensure a fair comparison, the ENS for non-configurable implementations has been consolidated.
[d] ○: No SCA countermeasures included ◑: Timing countermeasures ●: Timing+Power countermeasures
[e] Implementation for Kyber.
[f] This implementation study some operations independently.

ASIC synthesis flow. We used the Nangate45 cell library together with the Yosys/OpenSTA flow to estimate the ASIC area. We use a 500 MHz time closure, excluding the RAM bank from the synthesis. For the implementation of the unprotected hardware ML-KEM with $N_{\mathtt{BU}} = 1, 2$, and 4, an area size of 152.83kGE, 176.82kGE, and 222.65kGE are required, respectively. In contrast, the first-order and high-order protected implementations required around 385.67kGE and 580.56kGE, respectively. These results illustrate the independence of our design from technology.

5.2 Security Evaluation

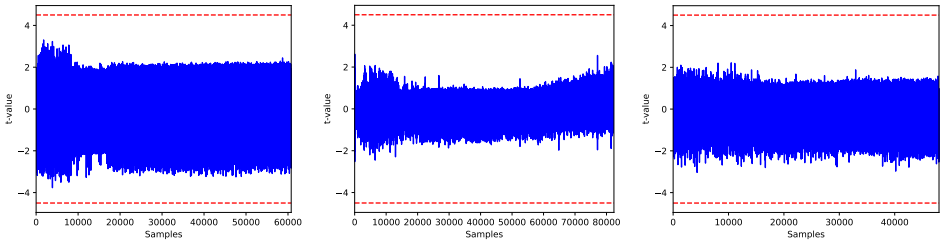
To assess the side-channel resistance of our ML-KEM hardware implementations, we performed non-specific fixed vs. random tests following the Test Vector Leakage Assessment (TVLA) methodology [SM15]. We focus on sensitive data across the three core operations: for key generation the seed d, for the encapsulation, the message m, and for the decapsulation, the analysis is limited to decryption, as conclusions from the encapsulation step can be applied to the re-encryption phase, considering as sensitive the dk_{PKE}. A key challenge in this evaluation is that the generation of the matrix $\hat{\mathbf{A}}$ depends on the public seed ρ, causing variability in execution time. Since ρ is public, we applied a small modification: during key generation, when the two seeds ρ and σ are generated, we perform the hash function to derive the secret σ while keeping ρ constant. This ensures constant-time



(a) First-order TVLA of unprotected for `ML-KEM-768.Encaps()` ($N_{BU} = 4$)

(b) Third-order TVLA without Keccak protection for `ML-KEM-768.Encaps()` ($N_{BU} = 4$)

Figure 9: TVLAs of unprotected and high-order protected implementations.



(a) `ML-KEM-768.KeyGen()` **(b)** `ML-KEM-768.Encaps()` **(c)** `K-PKE-768.Decrypt()`

Figure 10: First order TVLA for ML-KEM ($N_{BU} = 2$) with Keccak protection (`TI3`)

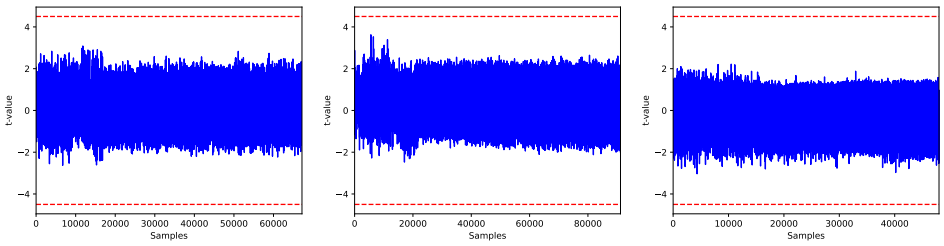
execution, and hence aligned traces as required in TVLA.

The experimental setup consists of a SAKURA-X board with the ML-KEM hardware module integrated with an UART interface to facilitate communication with a PC. The PC controls the test vectors and collects power traces using a PicoScope 5444D oscilloscope at 250 MSps, whereas the target clock frequency is set to 31.25 MHz. A preliminary analysis is presented in Figure 9, comparing an unprotected implementation with $N_{\texttt{BU}} = 4$ to a high-order protected implementation for `ML-KEM-768`, where the Keccak protection in the encapsulation process is not yet considered. For this evaluation, we collected $N = 1{,}000{,}000$ traces per set, applying a threshold of ± 4.5 at a significance level of $\alpha = 0.00001$. As expected, the unprotected version shows leakage throughout the entire operation. In contrast, the high-order protected version effectively protects the entire operation, with the exception of the Keccak, thereby demonstrating the practical effectiveness of the proposed high-order masking countermeasures.

A complete analysis is presented in Figures 10 and 11, showing all operations for both first- and high-order protected versions, now including the protection provided by Keccak. For the case of `ML-KEM.Decaps()` we only focus on the decryption part `K-PKE.Decrypt()` since the `K-PKE.Encrypt()` is included `ML-KEM.Encaps()`. Using $N = 1{,}000{,}000$ traces, we observe that both first- and high-order designs achieve an improved protection.

6 Conclusions

We present **`HOPE-MLKEM`**, a complete framework to seamlessly design high-order protection on hardware implementations for ML-KEM at all security levels and operations. Our results show that robust masking countermeasures can be systematically integrated without incurring high costs in area or performance. `HOPE-MLKEM` enables the comprehensive explo-



(a) `ML-KEM-768.KeyGen()` **(b)** `ML-KEM-768.Encaps()` **(c)** `K-PKE-768.Decrypt()`

Figure 11: Third order TVLA for ML-KEM ($N_{BU} = 4$) with Keccak protection (`DOM`)

ration of trade-offs between performance, resource efficiency, and side-channel protection, offering the community a modular, flexible and versatile framework for future research and deployment.

Future work includes investigating countermeasures against fault injection attacks and developing optimized modules for protected Keccak implementations. In addition, this work lays the foundation for extending our methodology to other post-quantum schemes, such as ML-DSA. It is also of interest to explore the integration of the module (or a submodule) as an ISA extension for RISC-V cores.

Acknowledgements

This research was supported by QUBIP Project with Grant Agreement No. 101119746 under the EU Horizon Europe research and innovation programme and by the international collaboration under TECHNOQUANTUM project (reference ILINK24018) from i-LINK 2024 CSIC programme.

References

[BBM+18] Davide Bellizia, Simone Bongiovanni, Pietro Monsurrò, Giuseppe Scotti, Alessandro Trifiletti, and Francesco Bruno Trotta. Secure Double Rate Registers as an RTL Countermeasure Against Power Analysis Attacks. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 26(7):1368–1376, 2018.

[BC22] Olivier Bronchain and Gaëtan Cassiers. Bitslicing arithmetic/boolean masking conversions for fun and profit with application to lattice-based KEMs. *IACR TCHES*, 2022(4):553–588, 2022.

[BGR+21] Joppe W. Bos, Marc Gourjon, Joost Renes, Tobias Schneider, and Christine van Vredendaal. Masking Kyber: First- and higher-order implementations. *IACR TCHES*, 2021(4):173–214, 2021.

[BNAMK21] Mojtaba Bisheh-Niasar, Reza Azarderakhsh, and Mehran Mozaffari-Kermani. Instruction-set accelerated implementation of crystals-kyber. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 68(11):4648–4659, 2021.

[CRNTCA] Eros Camacho-Ruiz, Pablo Navarro-Torrero, and Alejandro Cabrera-Aldaya. HOPE-MLKEM. https://github.com/HWSec-CSIC/hope-mlkem.

[DEM18] Thomas De Cnudde, Maik Ender, and Amir Moradi. Hardware masking, revisited. *IACR TCHES*, 2018(2):123–148, 2018.

[DMG23] Viet Ba Dang, Kamyar Mohajerani, and Kris Gaj. High-Speed Hardware Architectures and FPGA Benchmarking of CRYSTALS-Kyber, NTRU, and Saber. *IEEE Transactions on Computers*, 72(2):306–320, 2023.

[FSS20] Tim Fritzmann, Georg Sigl, and Johanna Sepúlveda. RISQ-V: Tightly coupled accelerators for post-quantum cryptography. *IACR TCHES*, 2020(4):239–280, 2020.

[GLK22] Wenbo Guo, Shuguo Li, and Liang Kong. An Efficient Implementation of KYBER. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 69(3):1562–1566, 2022.

[GMK16] Hannes Gross, Stefan Mangard, and Thomas Korak. Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order. Cryptology ePrint Archive, Report 2016/486, 2016.

[HKL+22] Daniel Heinz, Matthias J. Kannwischer, Georg Land, Thomas Pöppelmann, Peter Schwabe, and Amber Sprenkels. First-order masked Kyber on ARM Cortex-M4. Cryptology ePrint Archive, Report 2022/058, 2022.

[JD23] Yanning Ji and Elena Dubrova. A side-channel attack on a masked hardware implementation of CRYSTALS-kyber. Cryptology ePrint Archive, Report 2023/1084, 2023.

[JGCS21] Arpan Jati, Naina Gupta, Anupam Chattopadhyay, and Somitra Kumar Sanadhya. A configurable Crystals-Kyber hardware implementation with side-channel protection. Cryptology ePrint Archive, Report 2021/1189, 2021.

[JGCS24] Arpan Jati, Naina Gupta, Anupam Chattopadhyay, and Somitra Kumar Sanadhya. A Configurable CRYSTALS-Kyber Hardware Implementation with Side-Channel Protection. *ACM Trans. Embed. Comput. Syst.*, 23(2), March 2024.

[JNWD24] Sönke Jendral, Kalle Ngo, Ruize Wang, and Elena Dubrova. Breaking SCA-Protected CRYSTALS-Kyber with a Single Trace. In *2024 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 70–73, 2024.

[JWN+22] Yanning Ji, Ruize Wang, Kalle Ngo, Elena Dubrova, and Linus Backlund. A side-channel attack on a hardware implementation of CRYSTALS-Kyber. Cryptology ePrint Archive, Report 2022/1452, 2022.

[KJSL24] Hyunseon Kim, Haesung Jung, Ardianto Satriawan, and Hanho Lee. A Configurable ML-KEM/Kyber Key-Encapsulation Hardware Accelerator Architecture. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 71(11):4678–4682, 2024.

[KMSH24] Dur E Shahwar Kundi, Jose M. Bermudo Mera, Pierre-Yves Strub, and Michael Hutter. High-Performance NTT Hardware Accelerator to Support ML-KEM and ML-DSA. In *Proceedings of the 2024 Workshop on Attacks and Solutions in Hardware Security*, ASHES '24, page 100–105, New York, NY, USA, 2024. Association for Computing Machinery.

[KNAH22] Tendayi Kamucheka, Alexander Nelson, David Andrews, and Miaoqing Huang. A masked pure-hardware implementation of Kyber cryptographic algorithm. Cryptology ePrint Archive, Report 2022/1547, 2022.

[KRSS19] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4. Cryptology ePrint Archive, Report 2019/844, 2019.

[LFK+19] Weiqiang Liu, Sailong Fan, Ayesha Khalid, Ciara Rafferty, and Máire O'Neill. Optimized Schoolbook Polynomial Multiplication for Compact Lattice-Based Cryptography on FPGA. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 27(10):2459–2463, 2019.

[LS19] Vadim Lyubashevsky and Gregor Seiler. NTTRU: Truly fast NTRU using NTT. *IACR TCHES*, 2019(3):180–201, 2019.

[MJB+24] Michail Moraitis, Yanning Ji, Martin Brisfors, Elena Dubrova, Niklas Lindskog, and Håkan Englund. Securing CRYSTALS-Kyber in FPGA Using Duplication and Clock Randomization. *IEEE Design & Test*, 41(5):7–16, 2024.

[MLK24] Module-Lattice-Based Key-Encapsulation Mechanism Standard. National Institute of Standards and Technology, NIST FIPS PUB 203, U.S. Department of Commerce, August 2024.

[MS16] Santos Merino Del Pozo and François-Xavier Standaert. A note on the security of threshold implementations with $d + 1$ input shares. Cryptology ePrint Archive, Report 2016/420, 2016.

[Nat24] National Institute of Standards and Technology. Post-Quantum Cryptography, 2024. Accessed: 2024-11-10.

[NDMZ+21] Pietro Nannipieri, Stefano Di Matteo, Luca Zulberti, Francesco Albicocchi, Sergio Saponara, and Luca Fanucci. A RISC-V Post Quantum Cryptography Instruction Set Extension for Number Theoretic Transform to Speed-Up CRYSTALS Algorithms. *IEEE Access*, 9:150798–150808, 2021.

[NKK+23] Ziying Ni, Ayesha Khalid, Dur-e-Shahwar Kundi, Maire ONeill, and Weiqiang Liu. HPKA: A High-Performance CRYSTALS-Kyber Accelerator Exploring Efficient Pipelining . *IEEE Transactions on Computers*, 72(12):3340–3353, December 2023.

[NKLO25] Ziying Ni, Ayesha Khalid, Weiqiang Liu, and Máire O'Neill. A Highly Hardware Efficient ML-KEM Accelerator with Optimised Architectural Layers. *ACM Trans. Embed. Comput. Syst.*, 24(2), January 2025.

[NRR06] Svetla Nikova, Christian Rechberger, and Vincent Rijmen. Threshold implementations against side-channel attacks and glitches. In Peng Ning, Sihan Qing, and Ninghui Li, editors, *ICICS 06*, volume 4307 of *LNCS*, pages 529–545. Springer, Berlin, Heidelberg, December 2006.

[Rep15] Oscar Reparaz. A note on the security of higher-order threshold implementations. Cryptology ePrint Archive, Report 2015/001, 2015.

[Saa24] Markku-Juhani O. Saarinen. Accelerating SLH-DSA by two orders of magnitude with a single hash unit. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 276–304. Springer, Cham, August 2024.

[SH24] Aein Rezaei Shahmirzadi and Michael Hutter. Efficient boolean-to-arithmetic mask conversion in hardware. *CiC*, 1(3):46, 2024.

[SM15] Tobias Schneider and Amir Moradi. Leakage assessment methodology - A clear roadmap for side-channel evaluations. In Tim Güneysu and Helena Handschuh, editors, *CHES 2015*, volume 9293 of *LNCS*, pages 495–513. Springer, Berlin, Heidelberg, September 2015.

[SSM+23] Ardianto Satriawan, Infall Syafalni, Rella Mareta, Isa Anshori, Wervyan Shalannanda, and Aleams Barra. Conceptual Review on Number Theoretic Transform and Comprehensive Review on Its Implementations. *IEEE Access*, 11:70288–70316, 2023.

[TAMK25] Bardia Taghavi, Reza Azarderakhsh, and Mehran Mozaffari Kermani. ParallelNTT: Maximizing Performance of Forward and Inverse NTT on FPGA for ML-DSA and ML-KEM. In *Proceedings of the Great Lakes Symposium on VLSI 2025*, GLSVLSI '25, page 372–378, New York, NY, USA, 2025. Association for Computing Machinery.

[WBD23] Ruize Wang, Martin Brisfors, and Elena Dubrova. A side-channel attack on a bitsliced higher-order masked CRYSTALS-kyber implementation. Cryptology ePrint Archive, Report 2023/1042, 2023.

[XL21] Yufei Xing and Shuguo Li. A compact hardware implementation of CCA-secure key exchange mechanism CRYSTALS-KYBER on FPGA. *IACR TCHES*, 2021(2):328–356, 2021.

[XWT25] Dejun Xu, Kai Wang, and Jing Tian. A Hardware-Friendly Shuffling Countermeasure Against Side-Channel Attacks for Kyber. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 72(3):504–508, 2025.

[YMÖS21] Ferhat Yaman, Ahmet Can Mert, Erdinç Öztürk, and Erkay Savaş. A hardware accelerator for polynomial multiplication operation of CRYSTALS-KYBER PQC scheme. Cryptology ePrint Archive, Report 2021/485, 2021.

[ZLL+21] Cong Zhang, Dongsheng Liu, Xingjie Liu, Xuecheng Zou, Guangda Niu, Bo Liu, and Quming Jiang. Towards Efficient Hardware Implementation of NTT for Kyber on FPGAs. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5, 2021.

[ZPM+23] Yiqiang Zhao, Shijian Pan, Haocheng Ma, Ya Gao, Xintong Song, Jiaji He, and Yier Jin. Side Channel Security Oriented Evaluation and Protection on Hardware Implementations of Kyber. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 70(12):5025–5035, 2023.

[ZSS+21] Sara Zarei, Aein Rezaei Shahmirzadi, Hadi Soleimany, Raziye Salarifard, and Amir Moradi. Low-latency keccak at any arbitrary order. Cryptology ePrint Archive, Report 2021/892, 2021.

[ZYHK24] Jipeng Zhang, Yuxing Yan, Junhao Huang, and Çetin Kaya Koç. Optimized software implementation of keccak, kyber, and dilithium on RV32,64IMBV. Cryptology ePrint Archive, Report 2024/1515, 2024.